Internal Maps Platform Development Plan

Bangladesh-only basemap + 15M business POIs + 5M delivery events
Node.js + PostgreSQL/PostGIS + Elasticsearch

Date: February 08, 2026

Audience: Product, Engineering, and Infrastructure teams

1. Executive Summary

This document describes the end-to-end development plan for an internal maps platform that will be used across Bangladesh to visualize and maintain a large geospatial database (approximately 15 million business locations and 5 million delivery locations). The platform targets smooth user experience up to zoom level 18 in dense urban areas, while supporting daily POI edits by field sales representatives with full auditability.

Key design decisions:

1. Use a static Bangladesh basemap (OpenStreetMap-derived) packaged as PMTiles and served as a single file with HTTP range requests.
2. Store business POIs in PostgreSQL + PostGIS as the source of truth; track all edits in an append-only history table.
3. Serve map overlays as vector tiles (MVT/PBF) from PostGIS: clusters at low zoom (z0–z15) and raw points at high zoom (z16–z18).
4. Treat delivery locations as time-series events (partitioned by time) and visualize primarily as density/heatmap layers with filters.
5. Provide fast search and autocomplete via Elasticsearch, synced reliably from Postgres using an outbox + worker pattern.
6. Use caching (Nginx) and connection pooling (PgBouncer) to support ~1000 users efficiently.

2. Target Architecture

The platform is designed as a layered system: a static basemap, dynamic overlay tiles, a transactional API, and search. This keeps the basemap fast and predictable while enabling frequent updates to business locations.

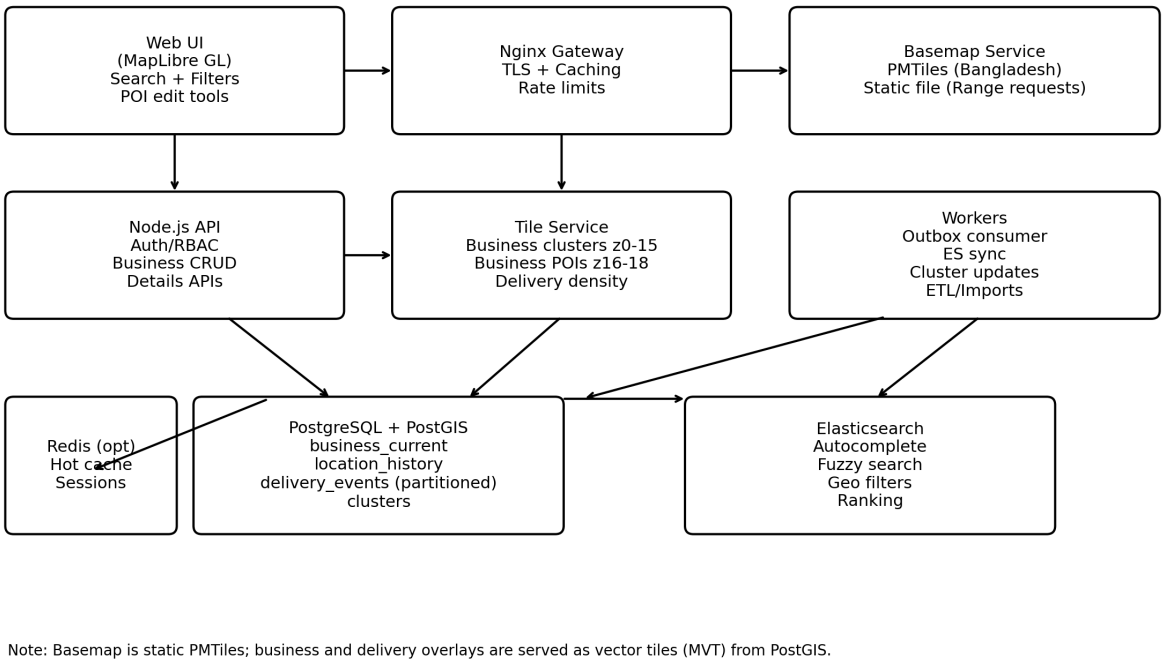

Figure 1 — High-level architecture (basemap + overlays + API + search).

2.1 Components

Web UI (MapLibre GL)

7. Interactive map with smooth pan/zoom to z18.
8. Layer toggles (businesses, deliveries), filters, and search.
9. POI edit tools (move pin, confirm large move, add note).

Nginx Gateway

10. TLS termination, caching for tile endpoints, and request limits.
11. Routes to basemap file server, vector tile service, and Node.js APIs.

Basemap Service (PMTiles)

12. Bangladesh-only vector basemap stored as a single PMTiles file.
13. Served as static content; clients fetch only required byte ranges.

Node.js API

14. Authentication and role-based access control (RBAC).

15. Business CRUD, POI movement workflow, and detail endpoints.
16. Delivery ingestion endpoints (batch/stream) and analytics endpoints.

Tile Service

17. Produces MVT/PBF tiles for business clusters, business points, and delivery density.
18. Can be implemented using Tegola (recommended) or a Node.js tile endpoint using ST_AsMVT.

Workers (Outbox consumers)

19. Reliable sync to Elasticsearch and incremental cluster updates.
20. ETL tasks (imports, reindex/rebuild), scheduled maintenance jobs.

PostgreSQL + PostGIS

21. Source of truth for POIs and delivery events.
22. Spatial indexing, partitioning for deliveries, and audit history.

Elasticsearch

23. Fast search, autocomplete, and filtering over business attributes.
24. Geo search (nearby, within radius/polygon) for discovery workflows.

3. Map Layers and Zoom Strategy

The map supports zoom level 18 for precise placement in dense regions. To keep tile size and rendering cost under control, low zoom levels must use clustering or density layers; raw points are only returned at high zoom.

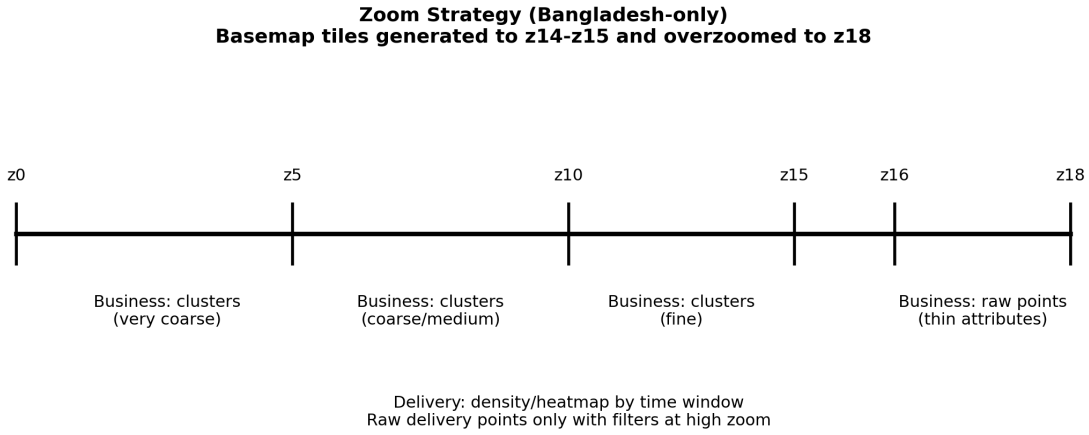

Figure 2 — Zoom strategy for business POIs and delivery events.

3.1 Basemap

Generate Bangladesh basemap vector tiles to z14–z15 and allow client overzoom to z18. This provides a Google-like experience without producing an excessively large tileset.

3.2 Business POIs (15M)

25. z0–z15: serve business clusters (pre-aggregated) instead of raw points.
26. z16–z18: serve raw business points with thin attributes (id, category, priority).
27. On click/hover: fetch full details via Node.js API using the business id.

3.3 Delivery Locations (5M)

28. Primary visualization: density/heatmap (aggregated grid) with time range controls (e.g., today, last 7 days).
29. Optional raw points at z16–z18 only when the user applies filters (time window, route, SKU, territory).
30. Delivery data stored as partitioned events; never treat deliveries as permanent POIs.

4. Data Model (PostgreSQL + PostGIS)

The data model separates the current state of each business from the full edit history. This keeps reads fast for tiles and APIs while preserving an append-only audit trail.

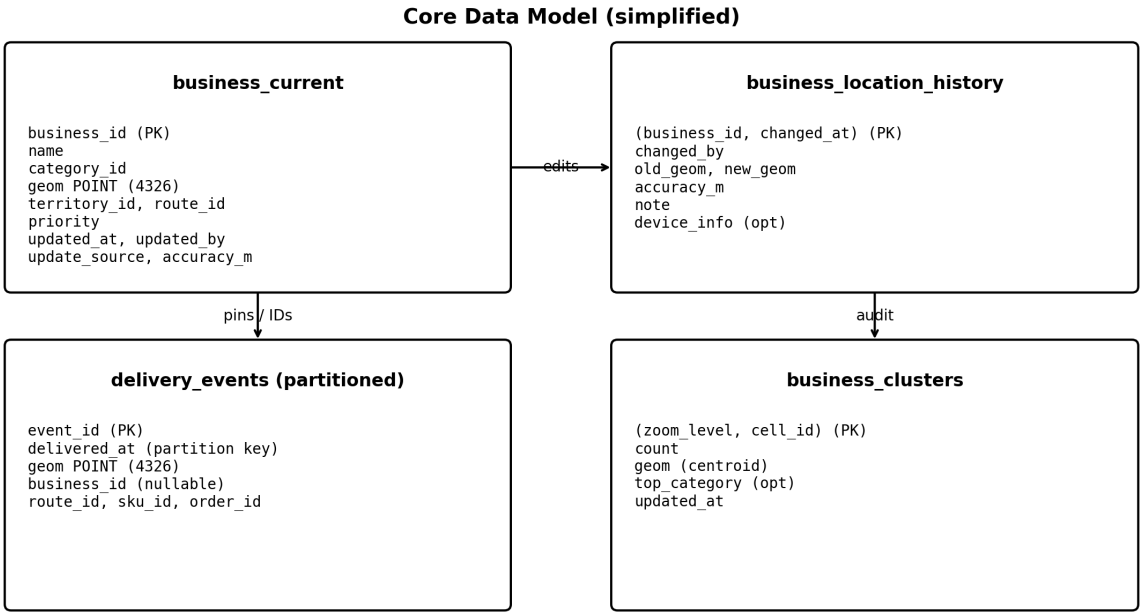

Figure 3 — Simplified core data model.

4.1 Core tables

Business current snapshot:

31. business_current: one row per business (15M). Includes name, category, current geometry, and last update metadata.
32. GiST index on geom + supporting B-tree indexes for common filters (category, route, territory, updated_at).

Business history (audit):

33. business_location_history: append-only records for every location change.
34. Stores old/new geometry, accuracy, rep/user, timestamp, and optional note/device info.

Deliveries (events):

35. delivery_events: partitioned by time (monthly recommended).
36. Indexes per partition: delivered_at, GiST(geom), and optional route_id/sku_id indexes for frequent filters.

Pre-aggregations:

37. business_clusters: precomputed grid/H3 clusters for z0–z15. Stores cell id, count, centroid, and updated_at.
38. delivery_density: aggregated density by grid cell + time window (can be computed on demand and cached, or precomputed for common windows).

5. Search (Elasticsearch) and Sync

Elasticsearch powers fast autocomplete and fuzzy search. Postgres remains the source of truth. A reliable sync mechanism keeps Elasticsearch and cluster tables up to date as reps edit POIs daily.

5.1 Index design (business_search)

39. Core fields: business_id, name (analyzed), address_text (analyzed), category_id, route_id, territory_id, updated_at, location (geo_point).
40. Autocomplete: use a dedicated autocomplete field strategy (e.g., search-as-you-type or edge n-grams) sized for 15M docs.
41. Search flow: UI query -> Node -> Elasticsearch -> list of IDs -> Node fetches details from Postgres.

5.2 Outbox + worker pattern

42. Node writes business updates to Postgres and inserts an outbox event in the same transaction.
43. Workers consume outbox events to: (1) update Elasticsearch documents, (2) update cluster cells affected by the move, (3) trigger cache invalidation/versioning as needed.
44. This prevents drift: if Elasticsearch is down, events retry until processed.

6. Tile Service Design

The tile service returns Mapbox Vector Tiles (MVT) for business overlays and delivery density. To avoid heavy payloads, tiles include only the fields required for rendering; full details are retrieved separately.

6.1 Recommended approach

45. Use Tegola to serve MVT from PostGIS for business clusters and high-zoom points.
46. Serve basemap PMTiles separately (static file) to decouple basemap from live POI updates.

47. Add Nginx caching for tile endpoints; cache clusters longer than raw points.

6.2 Tile payload rules

48. Raw points (z16–z18): return only id, category, and optionally priority; details fetched via API.
49. Safety cap per tile (e.g., 5,000 features). If exceeded: show clusters or show only top N by priority.
50. Cluster tiles: keep features per tile low; include count and centroid only (optional top-category summary).

7. POI Update Workflow and Data Quality Controls

Since business locations are updated daily by reps, the platform must prevent accidental drift, support review, and keep a complete audit trail.

7.1 Location update API

51. Endpoint: POST /business/{id}/location with lat/lon, accuracy, source, and optional note.
52. Write path: update business_current + append business_location_history + create outbox event.
53. Return: updated business summary + flags (e.g., low accuracy, large move requires review).

7.2 Validation rules

54. Large move confirmation: if moved > 200m (configurable), require explicit confirm + note.
55. Accuracy gating: if GPS accuracy is poor, mark the update as low confidence.
56. Suspicious edit detection: flag reps with too many large moves per day, repeated ping-pong moves, or improbable patterns.
57. Admin tools: rollback to a prior location and review queue for flagged updates.

8. Delivery Ingestion and Analytics

58. Ingestion: batch (daily) or streaming. Write to time partitions. Record business_id when matched; otherwise keep raw location.
59. Visualization: default density layer with time slider; show raw deliveries only with filters at high zoom.

60. Analytics examples: deliveries by route/territory/day, hotspot detection, SLA tracking, and exception monitoring.

9. Non-Functional Requirements

9.1 Performance targets

61. Smooth map interaction at z18 for dense regions; prefer GPU rendering (vector tiles) over raster.
62. Hot-tile latency target: p95 < 200 ms from cache; cold-tile p95 < 800 ms under normal load.
63. Support ~1000 users via caching + pooling; avoid direct 1000-connection pressure on Postgres.

9.2 Reliability and operations

64. PgBouncer for Postgres connection pooling; separate worker pools for heavy jobs.
65. Backups: Postgres PITR + periodic snapshots; PMTiles versioning; ES snapshots or rebuild-from-Postgres strategy.
66. Observability: metrics for tile latency, cache hit ratio, DB slow queries, worker lag, and ES query latency.

9.3 Security

67. RBAC: viewer vs editor vs admin; territory/route-level scoping if required.
68. Full audit log for POI edits; exportable reports for supervisors.
69. TLS everywhere; rate limits; optional SSO/LDAP integration.

10. Development Phases and Deliverables

Phase 1 — MVP (Basemap + Business POIs)

1. Basemap (Bangladesh PMTiles) served via Nginx.
2. Postgres schema: business_current + business_location_history.
3. Tile service: business clusters (z0–z15) + raw points (z16–z18).
4. Web UI: Map, layers, clustering, click-to-details.
5. Search: initial Elasticsearch index + autocomplete; search results zoom/pan.
6. POI update workflow: move pin + audit history.

Phase 2 — Scale (15M updates + 1000 users)

7. Outbox + workers for ES sync and cluster updates.
8. PgBouncer + Nginx caching policies tuned and validated under load.
9. Admin tools: flagged-move review queue + rollback.
10. Monitoring dashboards and alerting for tile latency, DB load, worker lag.

Phase 3 — Deliveries (Events + Density)

11. Partitioned delivery_events storage and ingestion pipeline.
12. Delivery density tiles + UI filters (time window, route, SKU, territory).
13. Optional raw delivery points at high zoom with filters.
14. Analytics endpoints and dashboards.

Phase 4 — Data Quality Automation

15. Confidence scoring, anomaly detection, and supervisor workflows.
16. Boundary tagging (district/upazila/ward) via spatial joins for reporting.
17. Operational tooling: reindex, rebuild clusters, bulk imports, and archive policies.

Appendix A — Single-Server Deployment (initial)

Given the available server (256 GB RAM, dual Xeon Platinum 8164, NVMe storage), start with a single-host deployment:

70. Postgres/PostGIS on NVMe; PgBouncer in front; dedicated worker connections.

71. Elasticsearch on NVMe; heap ~31 GB; OS cache uses remaining RAM.

72. Nginx for static PMTiles and tile/API caching.

73. Tile service (Tegola) and Node API as separate processes/containers.

74. Optional Redis for sessions/hot caches and rate limiting.